Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

총정리
직선 하나에서 RAG까지

10월 6일부터 16일까지 다룬 3장~25장을 한 흐름으로 다시 봅니다.
장마다 "핵심 한 줄"을 모으고, 헷갈리기 쉬운 것을 짝지어 비교하고, 종합 문제로 확인합니다.

오늘 다루는 것

1. 큰 그림: 다섯 단계로 본 전체 흐름
2. 장별 핵심 한 줄 모음
3. 한눈에 보는 비교표: 모델의 계보, 입력과 출력의 모양, 헷갈리는 짝
4. 종합 확인 문제

끝나고 나면 할 수 있는 것

1. "왜 이 기술이 나왔는가"를 앞 기술의 한계와 이어서 설명할 수 있다.
2. 문제를 보면 어떤 모양의 입력과 출력, 어떤 모델이 필요한지 고를 수 있다.
3. 각 장의 자료에서 다시 볼 곳을 찾아갈 수 있다.

2026년 10월 19일 월요일 총정리
교재: 『딥 러닝 파이토치 교과서 - 입문부터 LLM 파인튜닝까지』(브라이스 유, 안상준), https://wikidocs.net/book/2788
각 장의 절별 출처는 장별 자료에 적혀 있습니다.

큰 그림

2주 동안 배운 것은 하나의 질문에 대한 답이 차례로 바뀌어 온 이야기입니다. "글을 어떻게 숫자로 바꾸고, 그 숫자에서 어떻게 뜻을 읽어 낼 것인가." 앞 단계의 한계가 다음 단계가 나온 이유입니다.

다섯 단계

① 학습의 원리
3~8장
손실, 경사하강법, 신경망

② 글을 숫자로
9~12장, 15장
토큰, 벡터, 임베딩

③ 순서를 읽는 모델
13~17장
RNN, seq2seq, 어텐션

④ 어텐션만으로
18장
트랜스포머

⑤ 미리 배운 모델을 쓴다
19~25장
BERT, GPT, RAG

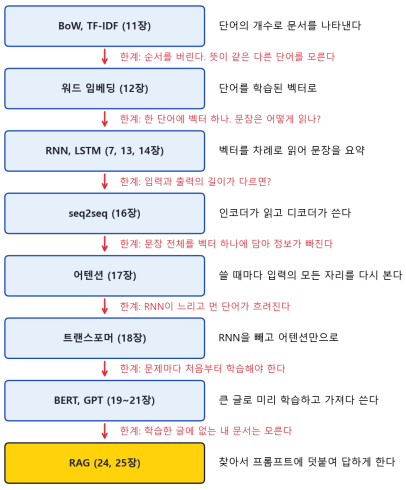


그림 1 무엇이 부족해서 다음 것이 나왔는가

1 다섯 단계로 본 전체 흐름

핵심 한 줄

모델은 "예측 → 손실 → 기울기 → 고치기"로 배우고, 글은 토큰과 벡터로 바뀌어 모델에 들어가며, 순서를 읽는 방법이 RNN에서 어텐션으로 바뀐 뒤에는 미리 학습한 큰 모델을 가져다 쓰는 것이 표준이 되었다.

① 학습의 원리 (3~8장)

정리

모든 장의 모델이 같은 네 단계로 배웁니다. 3장의 직선부터 21장의 GPT 파인 튜닝까지 한 번도 바뀌지 않았습니다.

$$\text{예측 } \hat{y} = f_\theta(x) \;\rightarrow\; \text{손실 } L(\hat{y}, y) \;\rightarrow\; \text{기울기 } \frac{\partial L}{\partial \theta} \;\rightarrow\; \text{고치기 } \theta \leftarrow \theta - \alpha \frac{\partial L}{\partial \theta}$$

기호	뜻	처음 나온 곳
θ	모델의 파라미터(가중치와 편향)	3장의 W, b
L	손실: 예측이 정답에서 얼마나 틀렸는지	3장 MSE, 4·5장 크로스 엔트로피
α	학습률: 한 번에 얼마나 고칠지	3장

문제	출력층	손실	장
숫자 맞히기 (회귀)	그대로	평균 제곱 오차	3
둘 중 하나	시그모이드	이진 크로스 엔트로피	4
여럿 중 하나	소프트맥스	크로스 엔트로피	5

층을 쌓으면(6장) 직선으로 못 푸는 문제를 풀 수 있고, 기울기는 역전파로 구합니다. 순서가 있는 데이터에는 RNN(7장), 이웃한 것끼리의 무늬에는 CNN(8장)을 씁니다.

② 글을 숫자로 (9~12장, 15장)

정리

단계	하는 일	한계 → 다음 단계
토큰화, 정제 (9장)	글을 토큰으로 자르고 번호를 붙인다	번호에는 뜻이 없다
BoW, TF-IDF (11장)	문서를 단어의 개수(가중치)로 나타낸다	순서를 버린다. 같은 뜻의 다른 단어를 모른다
워드 임베딩 (12장)	단어를 짧은 실수 벡터로. 비슷한 단어는 가까운 벡터	한 단어에 벡터 하나. 문맥을 모른다
서브워드 (15장)	단어를 더 작은 조각으로 쪼갠다	처음 보는 단어(OOV) 문제를 푼다
문맥 임베딩 (12-08, 19장)	같은 단어도 문장에 따라 다른 벡터	ELMo → BERT

언어 모델(10장)은 "앞 단어들로 다음 단어의 확률을 내는 모델"입니다. 통계로 세던 것(N-gram)이 신경망으로 바뀌었을 뿐, 21장의 GPT가 하는 일이 바로 이것입니다.

③ 순서를 읽는 모델 (13~17장)

정리

문제의 모양	예	모델의 모양	장
문장 → 클래스 하나 (다대일)	리뷰 감성 분류	RNN의 마지막 은닉 상태 → 선형층	13
문장 → 토큰마다 클래스 (다대다)	개체명 인식	양방향 LSTM의 모든 은닉 상태 → 선형층	14

문장 → 길이가 다른 문장	번역	인코더 + 디코더 (seq2seq)	16

seq2seq의 병목. 인코더가 문장 전체를 벡터 하나에 눌러 담아 디코더에 넘깁니다. 문장이 길면 정보가 빠집니다.

어텐션(17장). 디코더가 단어를 쓸 때마다 인코더의 모든 자리를 다시 보고, 지금 필요한 자리에 더 큰 가중치를 줍니다. 실습에서 BLEU가 0.2471에서 0.3007로 올랐습니다.

$$\text{가중치} = \text{softmax}(\text{질문 벡터와 각 자리 벡터의 점수}), \qquad \text{결과} = \sum(\text{가중치} \times \text{각 자리의 벡터})$$

④ 어텐션만으로 (18장)

정리

RNN은 한 단어씩 차례로 읽어 느리고, 멀리 떨어진 단어의 정보가 흐려집니다. 트랜스포머는 RNN을 빼고 어텐션만 남겼습니다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

부품	하는 일
포지셔널 인코딩	RNN이 없어 사라진 "순서" 정보를 벡터에 더해 준다
셀프 어텐션	문장 안의 모든 단어가 서로를 직접 참고한다
멀티 헤드	어텐션을 여러 개 나란히 돌려 서로 다른 관계를 본다
잔차 연결, 층 정규화	깊게 쌓아도 학습이 되게 한다
룩어헤드 마스크	디코더가 아직 쓰지 않은 뒤쪽 단어를 못 보게 가린다

$\sqrt{d_k}$로 나누는 이유는 17장 실습에서 직접 봤습니다. 내적 점수가 커지면 소프트맥스가 한 자리에 쏠립니다(디코더 단계의 88%에서 어텐션이 마지막 토큰 하나에 몰렸습니다).

⑤ 미리 배운 모델을 쓴다 (19~25장)

정리

	BERT (19장)	GPT (21장)
트랜스포머에서 가져온 것	인코더	디코더
사전 훈련 문제	가린 단어 맞히기 (양쪽을 본다)	다음 단어 맞히기 (앞만 본다)
쓰는 법	출력층을 붙여 파인 튜닝: 분류, 태깅, 독해. 문장 벡터로 검색	이어 쓰기. 크게 키우면 지시를 따르는 LLM

RAG(24, 25장). LLM은 학습한 글에 없는 것을 모릅니다. 모델을 다시 학습시키는 대신, 질문과 가까운 문서를 벡터 검색으로 찾아 프롬프트에 넣어 줍니다. 검색이 틀리면 답도 틀리므로, 리랭킹과 하이드(HyDE)로 검색을 고칩니다.

여기까지 오면 앞의 모든 장이 한 시스템 안에 들어 있습니다: 토큰화(9, 15장) → 임베딩(12, 19장) → 유사도 검색(11, 24장) → 트랜스포머 디코더의 생성(18, 21장).

2 장별 핵심 한 줄 모음

쓰는 법 각 장 자료의 절마다 맨 위에 있던 "핵심 한 줄"을 그대로 옮겼습니다. 읽고 뜻이 바로 떠오르지 않는 줄이 다시 볼 곳입니다.

3장 머신 러닝 입문 (10/6)

절	제목	핵심 한 줄
03-01	선형 회귀와 자동 미분	데이터에 가장 잘 맞는 직선(W, b)을 찾기 위해, **손실**로 틀린 정도를 재고 **경사하강법**으로 손실이 작아지는 쪽으로 조금씩 고친다.
03-02	다중 선형 회귀	입력이 여러 개($x_1, x_2, x_3, \ldots$)여도 방법은 똑같다. 입력마다 가중치($w_1, w_2, w_3, \ldots$)를 하나씩 두고, **행렬곱**으로 한 번에 계산한다.
03-03	nn.Module과 클래스로 구현하기	W와 b를 직접 만들고 MSE를 직접 쓰는 대신, 파이토치가 준비해 둔 **nn.Linear**, **F.mse_loss**, 그리고 모델 설계도인 **nn.Module 클래스**를 쓴다.
03-04	미니 배치와 데이터 로더	데이터가 많으면 한 번에 다 쓰지 않고 작은 묶음(**미니 배치**)으로 나눠 여러 번 학습한다. 나누고 섞는 일은 **DataLoader**가 해 준다.
03-05	벡터와 행렬 연산 복습하기	딥러닝의 데이터는 모두 **텐서**(숫자를 담는 상자)이고, 계산의 대부분은 **행렬 곱**(가로줄 × 세로줄을 곱해서 더하기)이다.

4장 로지스틱 회귀 (10/6)

절	제목	핵심 한 줄
04-01	로지스틱 회귀	로지스틱 회귀는 선형 점수 $Wx + b$를 시그모이드로 0~1의 확률로 바꿔 이진 분류를 하고, 이진 크로스 엔트로피를 최소화하도록 W와 b를 학습하는 모델이다.
04-02	nn.Module과 클래스로 구현하는 로지스틱 회귀	로지스틱 회귀는 nn.Linear(선형 점수)와 nn.Sigmoid(확률 변환)를 이은 모델이며, 이는 인공 신경망의 뉴런 하나와 같은 구조다.

5장 소프트맥스 회귀 (10/6)

절	제목	핵심 한 줄
05-01	원-핫 인코딩	원-핫 인코딩은 선택지 수만큼 칸을 만들고 정답 자리만 1, 나머지는 0으로 표시하는 방법이다.
05-02	소프트맥스 회귀 이해하기	소프트맥스 회귀는 선택지마다 점수를 내고, 소프트맥스로 합이 1인 확률로 바꾼 뒤, 크로스 엔트로피로 정답 확률이 커지도록 학습한다.
05-03	소프트맥스 회귀 다양한 방법으로 구현하기	크로스 엔트로피는 직접 쓰거나 F.cross_entropy 한 줄로 쓸 수 있고, F.cross_entropy는 소프트맥스까지 포함하므로 모델 출력(점수)을 그대로 넣는다.
05-04	소프트맥스 회귀로 MNIST 데이터 분류하기	28×28 손글씨를 784칸으로 펴서 nn.Linear(784, 10) 하나로 숫자 10개의 점수를 내면, 은닉층 없이도 약 90% 이상을 맞힌다.

6장 인공 신경망 (10/7)

절	제목	핵심 한 줄
06-01	머신 러닝 기본 개념 정리하기	모델은 **분류**나 **회귀**를 하고, 정확도·정밀도·재현율로 평가하며, **활성화 함수**로 직선이 아닌 관계를 표현한다.
06-02	퍼셉트론	퍼셉트론은 입력에 가중치를 곱해 더하고 기준을 넘으면 1을 내는 인공 뉴런이다. 한 층으로는 XOR을 못 풀지만, 층을 쌓으면 풀 수 있다.

06-03	XOR 문제: 단층 퍼셉트론과 다층 퍼셉트론	같은 XOR 데이터를 파이토치로 학습시키면, 단층은 끝까지 못 풀고 은닉층을 쌓은 다층 퍼셉트론은 정확히 풀어낸다.
06-04	역전파	역전파는 출력의 오차를 뒤에서 앞으로 거슬러 보내며, 각 가중치가 오차에 얼마나 책임이 있는지(기울기)를 계산하는 방법이다.
06-05	다층 퍼셉트론으로 손글씨 분류하기	8×8 손글씨 그림을 64개 숫자로 펴서 다층 퍼셉트론에 넣으면, 0~9 중 어떤 숫자인지 분류할 수 있다.
06-06	다층 퍼셉트론으로 MNIST 분류하기	더 크고 유명한 손글씨 데이터 MNIST(28×28, 7만 장)를 훈련용과 테스트용으로 나누고, 미니 배치로 학습해 처음 보는 그림의 정확도로 평가한다.
06-07	과적합을 막는 방법들	과적합은 데이터 늘리기, 모델 줄이기, 가중치 규제, 드롭아웃으로 막는다.
06-08	기울기 소실과 폭주	층이 깊으면 역전파되는 기울기가 앞쪽에서 거의 0이 되거나(소실) 너무 커진다(폭주). ReLU, 좋은 초기화, 배치 정규화로 막는다.

7장 순환 신경망 (10/7)

절	제목	핵심 한 줄
07-01	순환 신경망(RNN)	RNN은 같은 셀을 시점마다 반복해서 쓰면서, 이전 시점의 은닉 상태(기억)를 현재 입력과 함께 계산해 다음 시점으로 넘기는 신경망이다.
07-02	LSTM과 GRU	바닐라 RNN은 시퀀스가 길어지면 앞쪽 정보를 잊는다(장기 의존성 문제). LSTM은 셀 상태와 세 게이트로 무엇을 지우고, 저장하고, 내보낼지 골라 긴 시퀀스에서도 기억을 지킨다.
07-03	문자 단위 RNN(Char RNN)	문자를 원-핫 벡터로 바꿔 시점마다 하나씩 넣고, 다 대 다 RNN이 시점마다 "다음 글자"의 확률을 내도록 크로스 엔트로피로 학습시킨다.

8장 합성곱 신경망 (10/7)

절	제목	핵심 한 줄
08-01	합성곱과 풀링	이미지는 가까운 픽셀끼리 관련이 깊으므로, CNN은 작은 커널로 주변 픽셀을 함께 계산하는 합성곱으로 특징을 뽑고, 풀링으로 그 결과를 요약한다.
08-02	CNN으로 MNIST 분류하기	CNN은 합성곱층과 풀링층을 반복해 이미지를 작지만 깊은(채널이 많은) 특성 맵으로 바꾸고, 이를 펼쳐 전결합층과 소프트맥스로 분류한다.
08-03	깊은 CNN으로 MNIST 분류하기	합성곱층과 전결합층을 더 쌓고 드롭아웃을 넣으면 더 복잡한 무늬를 배울 수 있지만, 층을 늘린다고 항상 좋아지지는 않는다.

9장 자연어 처리 기초 (10/8)

절	제목	핵심 한 줄
09-01	토큰화	토큰화는 글을 의미 있는 단위(토큰)로 자르는 일이고, 영어는 띄어쓰기와 규칙으로 대부분 되지만 한국어는 형태소 단위로 잘라야 한다.
09-02	텍스트 데이터의 정제와 정규화	정제는 노이즈를 지우는 일이고, 정규화는 표기가 다른 같은 단어를 하나로 맞추는 일이다.
09-03	불용어	불용어는 자주 나오지만 글의 내용을 구분하는 데 거의 도움이 안 되는 단어이고, 목록을 만들어 토큰에서 걸러 낸다.
09-04	정규 표현식	정규 표현식은 "숫자 3개-4개-4개"처럼 글자의 패턴을 기호로 적어, 그 패턴에 맞는 부분을 찾고, 자르고, 바꾸는 도구다.
09-05	딥 러닝을 위한 자연어 처리 전처리 실습	토큰화 → 단어 집합 → 정수 인코딩 → 패딩을 거치면, 길이가 제각각인 문장들이 (문장 수, 최대 길이) 모양의 정수 텐서가 된다.

10장 언어 모델 (10/8)

절	제목	핵심 한 줄
10-01	언어 모델이란?	언어 모델은 단어의 나열(문장)에 확률을 매기는 모델이고, 가장 흔한 방식은 앞 단어들을 보고 다음 단어의 확률을 예측하는 것이다.
10-02	통계적 언어 모델과 N-gram 언어 모델	통계적 언어 모델은 말뭉치에서 횟수를 세어 확률을 구하고, N-gram은 문맥을 바로 앞 N-1개 단어로 줄여서 셀 수 있는 경우를 늘린다.
10-03	펄플렉서티(PPL)	펄플렉서티는 언어 모델이 다음 단어를 고를 때 평균 몇 개 중에서 헷갈리는지를 나타내는 숫자이고, 낮을수록 좋다.

11장 텍스트 유사도 (10/12)

절	제목	핵심 한 줄
11-01	단어의 표현 방법	단어를 숫자로 나타내는 방법은, 단어 자체에 값을 붙이는 국소 표현과 주변 단어를 참고해 뜻을 벡터에 담는 분산 표현으로 나뉜다.
11-02	백 오브 워즈(Bag of Words)	Bag of Words는 단어의 순서를 버리고 각 단어가 나온 횟수만 세어, 문서 하나를 단어 집합 크기의 벡터 하나로 나타내는 방법이다.
11-03	DTM과 TF-IDF 행렬	DTM은 문서마다의 BoW를 쌓은 행렬이고, TF-IDF는 여러 문서에 흔히 나오는 단어의 값은 낮추고 특정 문서에만 나오는 단어의 값은 높인 행렬이다.
11-04	코사인 유사도를 이용한 추천 시스템	코사인 유사도는 두 벡터가 가리키는 방향이 얼마나 같은지를 재는 값이고, 문서 벡터에 쓰면 길이에 상관없이 내용이 비슷한 문서를 찾을 수 있다.
11-05	단어와 문서의 유사도를 구하는 다양한 방법	유클리드 거리는 두 점 사이의 직선 거리를, 자카드 유사도는 겹치는 단어의 비율을 재며, 무엇을 "비슷하다"고 볼지에 따라 방법을 고른다.

12장 텍스트 임베딩 (10/12)

절	제목	핵심 한 줄
12-01	NLP에서의 원-핫 인코딩	원-핫 인코딩은 단어 집합 크기의 벡터에서 그 단어의 번호 자리만 1로 두는 표현이고, 벡터가 너무 길고 단어 사이의 유사도를 나타내지 못한다는 한계가 있다.
12-02	워드 임베딩	워드 임베딩은 단어를 길이가 짧고 값이 실수인 밀집 벡터로 나타내는 방법이고, 그 값은 사람이 정하지 않고 데이터에서 학습한다.
12-03	워드투벡터(Word2Vec)	Word2Vec은 "비슷한 자리에 나오는 단어는 뜻이 비슷하다"는 가정 아래, 주변 단어로 중심 단어를 맞히는 (또는 그 반대의) 문제를 풀게 하면서 단어의 임베딩 벡터를 학습하는 방법이다.
12-04	Word2Vec 학습 및 시각화	gensim의 Word2Vec에 토큰화한 문장들을 넣으면 임베딩 행렬 (단어 수, 차원)이 만들어지고, 그 벡터로 비슷한 단어를 찾거나 2차원에 그려 볼 수 있다.
12-05	글로브와 패스트텍스트	GloVe는 말뭉치 전체에서 센 "함께 나온 횟수"를 임베딩에 반영하고, FastText는 단어를 글자 조각으로 쪼개 처음 보는 단어와 오타에도 벡터를 만들어 준다.
12-06	파이토치의 nn.Embedding()	임베딩 층은 단어의 번호로 표에서 한 행을 꺼내 오는 룩업 테이블이고, 그 표의 값은 신경망의 다른 가중치처럼 학습된다.
12-07	사전 훈련된 워드 임베딩	임베딩 층은 내 데이터로 처음부터 학습할 수도 있고, 방대한 글로 미리 학습된 벡터를 넣어 쓸 수도 있으며, 내 데이터가 적을 때는 뒤쪽이 유리하다.
12-08	엘모(ELMo)	ELMo는 미리 학습한 양방향 언어 모델의 각 층 출력을 가중합해, 같은 단어라도 문맥에 따라 다른 임베딩 벡터를 만든다.
12-09	단어 단위 RNN - 임베딩 사용	단어 단위 RNN은 단어의 번호를 임베딩 층으로 벡터로 바꿔 RNN에 넣고, 시점마다 선형층으로 다음 단어의 점수를 내는 신경망 언어 모델이다.

13장 텍스트 분류 (10/13)

절	제목	핵심 한 줄
13-01	RNN을 이용한 텍스트 분류	텍스트 분류는 글을 다 읽은 뒤 정답 하나를 내는 다 대 일 문제이고, RNN의 마지막 시점의 은닉 상태를 출력층에 넣어 클래스마다의 점수를 얻는다.
13-02	LSTM을 이용한 네이버 영화 리뷰 분류	리뷰를 정제, 토큰화, 정수 인코딩, 패딩으로 (리뷰 수, 30) 텐서로 만들고, 임베딩 → LSTM → 선형층 모델을 학습시키되 검증 손실이 가장 낮을 때의 모델을 저장해 쓴다.
13-03	GRU를 이용한 IMDB 리뷰 분류하기	같은 구조에서 LSTM을 GRU로, 데이터를 영어 리뷰로 바꾼 모델이며, 리뷰가 길어 뒤에 패딩이 많이 붙으면 어느 시점의 은닉 상태를 쓰는지가 학습에 큰 영향을 준다.
13-04	1D CNN을 이용한 IMDB 리뷰 분류	1D 합성곱은 커널이 문장 위를 단어 몇 개씩 훑으며 값을 내는 연산이고, 맥스 풀링으로 커널마다 가장 큰 값 하나만 남겨 문장을 벡터 하나로 요약한다.
13-05	사전 훈련된 임베딩을 이용한 성능 상승 시키기	임베딩 층을 무작위 값 대신 사전 훈련된 벡터로 시작하면, 같은 모델과 같은 데이터로도 더 빨리 더 높은 정확도에 도달한다.

14장 시퀀스 레이블링 (10/13)

절	제목	핵심 한 줄
14-01	시퀀스 레이블링	시퀀스 레이블링은 입력의 단어마다 레이블을 하나씩 붙이는 다 대 다 문제이고, 앞뒤 문맥을 모두 보기 위해 양방향 RNN의 모든 시점의 출력을 출력층에 넣는다.
14-02	양방향 LSTM을 이용한 개체명 인식	개체명 인식은 문장에서 사람·조직·장소 같은 이름을 찾아 유형을 붙이는 일이고, BIO 표기로 단어마다 태그를 붙인 데이터를 양방향 LSTM으로 학습한다.

15장 서브워드 토크나이저 (10/13)

절	제목	핵심 한 줄
15-01 ①	OOV 문제와 BPE	BPE는 글자 단위에서 시작해, 가장 자주 이웃해 나오는 두 조각을 하나로 합치는 일을 정해진 횟수만큼 반복해서 서브워드 단어 집합을 만든다.
15-01 ②	새 단어 쪼개기와 실제 토크나이저	처음 보는 단어는 글자로 나눈 뒤 학습 때 합친 순서대로 다시 합쳐서 쪼개며, 이렇게 하면 같은 크기의 단어 집합으로도 <UNK>가 거의 생기지 않는다.

16장 시퀀스투시퀀스 (10/14)

절	제목	핵심 한 줄
16-01	시퀀스투시퀀스	seq2seq는 인코더 RNN이 입력 문장을 읽어 컨텍스트 벡터 하나로 요약하고, 디코더 RNN이 그 벡터에서 출발해 <eos>가 나올 때까지 단어를 하나씩 만드는 구조다.
16-02	Seq2Seq를 이용한 번역기 구현하기	영어-프랑스어 문장 쌍을 인코더 입력, 디코더 입력(<sos> 붙임), 디코더 정답(<eos> 붙임)의 세 텐서로 만들고, LSTM 인코더와 디코더를 교사 강요로 학습시킨 뒤 한 단어씩 번역한다.
16-03	BLEU Score	BLEU는 기계 번역이 사람의 번역과 n-gram을 얼마나 공유하는지를 세어 0~1의 점수로 나타내며, 같은 단어의 반복과 너무 짧은 번역이 점수를 얻지 못하도록 두 가지 보정을 한다.

17장 어텐션 (10/14)

절	제목	핵심 한 줄
17-01	어텐션 메커니즘	어텐션은 디코더의 현재 은닉 상태(Query)로 인코더의 모든 은닉 상태(Key)와 점수를 내고, 소프트맥스로 비율을 정한 뒤, 그 비율로 인코더의 은닉 상태(Value)를 가중합해 예측에 쓰는 방법이다.
17-02	바다나우 어텐션	바다나우 어텐션은 디코더의 이전 시점의 은닉 상태를 Query로 쓰고, 작은 신경망으로 스코어를 구하며, 어텐션 값을 입력 임베딩과 이어 붙여 디코더 LSTM의 입력으로 넣는다.

17-03	어텐션 메커니즘을 이용한 번역기 구현	16장의 번역기에서 인코더의 모든 시점의 출력을 버리지 않고 디코더에 넘겨, 디코더가 시점마다 닷-프로덕트 어텐션으로 입력 문장을 다시 보게 하면 같은 데이터로도 번역이 나아진다.

18장 트랜스포머 (10/15)

절	제목	핵심 한 줄
18-01 ①	큰 그림: 왜 RNN을 버렸나	트랜스포머는 인코더-디코더 구조는 그대로 두고, 단어를 차례로 읽는 RNN 대신 모든 단어가 서로를 직접 참고하는 어텐션으로 층을 만들어 여러 개 쌓은 모델이다.
18-01 ②	포지셔널 인코딩	트랜스포머는 단어를 한꺼번에 읽어 순서를 모르므로, 위치마다 정해진 사인·코사인 값을 임베딩 벡터에 더해 "몇 번째 단어인지"를 알려 준다.
18-01 ③	셀프 어텐션	셀프 어텐션은 문장의 각 단어가 같은 문장의 모든 단어와 점수를 내어 비율을 정하고, 그 비율대로 다른 단어들의 정보를 섞어 "문맥이 반영된 자기 벡터"를 만드는 연산이다.
18-01 ④	멀티 헤드 어텐션과 패딩 마스크	멀티 헤드 어텐션은 작은 어텐션 여러 개를 서로 다른 가중치로 동시에 하고 결과를 이어 붙이는 방법이고, 패딩 마스크는 뜻 없는 <PAD>가 참고 대상이 되지 않도록 가리는 장치다.
18-01 ⑤	인코더 층 완성: FFNN, 잔차 연결, 층 정규화	인코더 층은 멀티 헤드 셀프 어텐션과 피드 포워드 신경망 두 서브층으로 이루어지고, 서브층마다 입력을 출력에 더한 뒤(잔차 연결) 값의 크기를 맞춰(층 정규화) 여러 층을 쌓아도 학습이 되게 한다.
18-01 ⑥	디코더: 한 단어씩 번역을 만든다	디코더는 뒤의 단어를 가린 셀프 어텐션(룩-어헤드 마스크)으로 지금까지 만든 번역을 읽고, 인코더-디코더 어텐션으로 입력 문장을 참고해 다음 단어의 확률을 낸다.

19장 BERT (10/16)

절	제목	핵심 한 줄
19-01	자연어 처리에서의 사전 훈련	사전 훈련은 레이블이 없는 방대한 글로 모델을 미리 학습시켜 두는 것이고, 그 대상이 단어 벡터(Word2Vec)에서 문맥을 읽는 모델 전체(ELMo, GPT, BERT)로 넓어져 왔다.
19-02	버트(BERT)	BERT는 트랜스포머 인코더를 쌓은 모델로, 단어·위치·세그먼트 임베딩의 합을 입력받아 토큰마다 문맥이 반영된 벡터를 내고, 마스크드 언어 모델과 다음 문장 예측으로 사전 훈련한 뒤 문제에 맞게 파인 튜닝해 쓴다.
19-03	BERT의 마스크드 언어 모델	사전 훈련된 BERT에 [MASK]가 든 문장을 넣으면, [MASK] 자리의 출력 벡터가 선형층과 소프트맥스를 거쳐 단어 집합 전체의 확률이 되고, 그중 확률이 높은 단어가 빈칸의 답이다.
19-04	BERT를 이용한 다음 문장 예측	문장 두 개를 [CLS] A [SEP] B [SEP]로 이어 넣으면, [CLS] 자리의 출력 벡터가 "B가 A 다음에 이어지는 문장인가"의 두 점수가 된다.
19-05	문장과 문서 임베딩을 위한 SBERT	SBERT는 문장 벡터끼리의 코사인 유사도가 뜻의 비슷한 정도가 되도록 BERT를 파인 튜닝한 모델이고, 문장 검색과 추천에 쓴다.

21장 GPT 학습하기 (10/16)

절	제목	핵심 한 줄
21-01	지피티(Generative Pre-trained Transformer, GPT)	GPT는 트랜스포머의 디코더 층만 쌓은 언어 모델로, 앞의 토큰들만 보고 다음 토큰을 맞히도록 사전 훈련되었으며, 구조는 거의 그대로 둔 채 크기를 키워 온 것이 지금의 거대 언어 모델이다.
21-02	GPT-2를 이용한 문장 생성	GPT는 다음 토큰의 확률만 낸다. 그 확률에서 토큰 하나를 고르고, 입력 뒤에 붙여 다시 넣는 일을 반복하는 것이 문장 생성이고, 어떻게 고르느냐에 따라 글이 달라진다.
21-03	GPT-2를 이용한 한국어 뉴스 감성 분류	GPT로 문장을 분류할 때는 문장 전체를 읽은 유일한 자리인 마지막 토큰의 벡터에 선형층을 붙이고, 나머지는 BERT의 파인 튜닝과 똑같이 학습시킨다.
21-04	GPT-2를 이용한 챗봇 파인 튜닝	질문과 답을 특별 토큰으로 구분해 한 줄의 글로 이어 붙이고 평소처럼 "다음 토큰 맞히기"로 학습시키면, GPT는 질문 뒤에 답을 이어 쓰는 챗봇이 된다.

24장 랭체인으로 만드는 RAG (10/16)

절	제목	핵심 한 줄
24-01	랭체인을 시작하기 전에	랭체인은 LLM 서비스에 필요한 부품(문서 로더, 분할기, 임베딩, 벡터 데이터베이스, 프롬프트, LLM)을 같은 문법으로 묶어 주는 도구일 뿐이고, 배워야 할 것은 도구가 아니라 그 부품들이 하는 일이다.
24-02	RAG 이해하기	문서를 미리 벡터로 바꿔 두고, 질문도 벡터로 바꿔 코사인 유사도가 높은 문서 몇 개를 찾아 질문과 함께 LLM에 넘기는 것이 RAG의 뼈대다.
24-03	랭체인의 두 가지 청킹 전략	긴 문서는 검색의 단위가 될 작은 조각(청크)으로 잘라야 하고, 자르는 방법에는 정해진 길이로 자르는 것과 뜻이 바뀌는 곳에서 자르는 것이 있다.
24-04	ChatOpenAI와 Memory	랭체인에서 LLM은 프롬프트를 받아 답을 돌려주는 부품이고, 프롬프트 템플릿과 | 로 이어 체인을 만든다. LLM은 지난 대화를 기억하지 못하므로, 기억은 지난 대화를 프롬프트에 다시 넣어 주는 것으로 만든다.
24-05	랭체인의 다양한 PDF 로더	PDF 로더는 PDF를 읽어 본문(page_content)과 출처 정보(metadata)를 가진 Document의 목록으로 바꿔 주고, 로더마다 속도와 뽑아내는 글, 메타데이터가 다르다.
24-06	벡터 데이터베이스	벡터 데이터베이스는 청크의 글, 벡터, 메타데이터를 함께 넣어 두고 질문과 가까운 청크를 찾아 주는 저장소로, 임베딩과 검색을 함수 한두 개로 대신해 준다.
24-07	RAG를 이용한 한국은행 금융 챗봇	RAG 챗봇은 "문서 읽기 → 전처리 → 벡터 데이터베이스에 넣기"를 미리 해 두고, 질문이 오면 "검색 → 프롬프트에 검색 결과와 질문 채우기 → LLM 호출"을 차례로 하는 함수 하나다.

25장 고급 RAG 기법 (10/16)

절	제목	핵심 한 줄
준비	공통 준비물: 벡터 데이터베이스와 LLM	세 실습이 같은 벡터 데이터베이스와 같은 LLM을 쓰므로, 한 파일(rag_base.py)에 만들어 두고 불러 쓴다.
25-01	LLM 기반의 리랭킹	리랭킹은 벡터 검색이 찾아온 후보를 더 정확한 방법으로 다시 채점해 순서를 바꾸는 것이고, LLM 기반 리랭킹은 그 채점을 LLM에게 프롬프트로 시킨다.
25-02	크로스 인코더 기반의 리랭킹	이중 인코더는 질문과 문서를 따로 읽어 벡터끼리 비교하고, 크로스 인코더는 둘을 한 입력으로 함께 읽어 관련성 점수 하나를 낸다. 앞의 것으로 빠르게 추리고 뒤의 것으로 정확히 다시 고른다.
25-03	검색 성능을 높이는 하이드(HyDE)	하이드는 질문을 바로 임베딩해 검색하지 않고, LLM에게 그 질문의 답처럼 보이는 가상의 문서를 먼저 쓰게 한 뒤 그 가상 문서를 임베딩해 검색한다.

3 한눈에 보는 비교표

핵심 한 줄 **모델을 고를 때는 이름보다 "무엇이 들어가서 무엇이 나오는가"를 먼저 본다.**

1) 입력과 출력의 모양

정리

n은 토큰 수, D는 벡터의 크기, $|V|$는 단어 집합의 크기, C는 클래스 수입니다.

부품	입력 → 출력	장
임베딩 층	$(n,) \to (n, D)$	12
RNN / LSTM	$(n, D) \to (n, H)$ 와 마지막 은닉 상태 $(H,)$	7, 13
문장 분류	$(n,) \to (C,)$	13
토큰 분류	$(n,) \to (n, C)$	14
트랜스포머 인코더 층, BERT	$(n, D) \to (n, D)$	18, 19
GPT (언어 모델)	$(n,) \to (n, \|V\|)$: 자리마다 다음 토큰의 점수	21
문장 임베딩 (SBERT)	문장 → $(D,)$	19-05
벡터 검색	질문 벡터 $(D,)$ → 가까운 문서 k개의 번호	24-06

2) 헷갈리는 짝

정리

짝	앞의 것	뒤의 것
소프트맥스 / 시그모이드	합이 1. 여럿 중 하나를 고른다	칸마다 0~1. 해당하는 것을 모두 고른다
BoW·TF-IDF / 임베딩	단어의 개수. 글자가 같아야 같다고 본다	학습된 벡터. 뜻이 비슷하면 가깝다
워드 임베딩 / 문맥 임베딩	단어마다 벡터 하나 (Word2Vec)	문장에 따라 달라지는 벡터 (ELMo, BERT)
RNN / 트랜스포머	차례로 읽는다. 순서 정보가 구조에 있다	한꺼번에 읽는다. 순서는 포지셔널 인코딩으로 넣는다
어텐션 / 셀프 어텐션	디코더가 인코더를 본다 (17장)	문장이 자기 자신을 본다 (18장)
BERT / GPT	인코더, 양방향, 빈칸 맞히기, 이해	디코더, 단방향, 다음 단어 맞히기, 생성
사전 훈련 / 파인 튜닝	레이블 없는 큰 글로 미리 학습	내 데이터로 조금 더 학습
파인 튜닝 / RAG	모델의 가중치를 바꾼다	가중치는 그대로, 프롬프트에 문서를 넣는다
이중 인코더 / 크로스 인코더	따로 읽어 벡터끼리 비교. 빠르다	함께 읽어 점수 하나. 정확하지만 느리다
그리디 / 샘플링	확률이 가장 높은 토큰. 늘 같은 글	확률에 따라 뽑는다. 매번 다른 글

3) 실습에서 얻은 숫자들

다시 보기 자료에 실린 출력에서 장마다 하나씩 골랐습니다. 숫자가 무엇을 뜻했는지 설명할 수 있으면 그 장을 이해한 것입니다.

장	숫자	뜻
14	토큰의 83.3%가 O 태그	모두 O로 찍어도 정확도 0.83. 정확도만 보면 속는다
15	단어 집합 8,000에서 UNK 12.8% → BPE 0%	서브워드로 쪼개면 처음 보는 단어가 사라진다
16 → 17	BLEU 0.2471 → 0.3007	어텐션을 붙이니 번역이 나아졌다
17	디코더 단계의 88%에서 어텐션이 마지막 토큰에 몰림	내적 점수가 크면 소프트맥스가 한쪽으로 쏠린다 → 18장의 $\sqrt{d_k}$
19	bank의 벡터 유사도 0.936 / 0.300	같은 단어도 문맥에 따라 다른 벡터 (문맥 임베딩)
19	날씨-주식 문장 유사도 0.695 → 0.301	BERT 평균 풀링 대신 SBERT를 써야 뜻이 반영된다
21	"근육이 커지기 위해서는" 다음 토큰 1등의 확률 0.02	언어 모델은 다음 토큰의 분포를 낼 뿐, 고르는 방법이 글을 정한다
24	HDRI 질문: 검색 없이 "고위험 가구"(틀림) → 검색 포함 "위험가구"(맞음)	RAG가 하는 일
25	벡터 검색 8위 청크가 크로스 인코더 리랭킹 2위로	함께 읽는 모델이 더 정확히 고른다

4 종합 확인 문제

푸는 법

장 하나가 아니라 여러 장을 이어야 답할 수 있는 문제들입니다. 한 명씩 돌아가며 풀고, 막히면 괄호 안의 장으로 돌아갑니다.

확인 문제

종합 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 모델이 배우는 네 단계를 순서대로 쓰면 예측 → _____ → 기울기 → 고치기이다. (3장) 이름 _____

2 세 클래스 중 하나를 고르는 문제의 출력층에는 시그모이드를 쓴다. (O / X) (5장) 이름 _____

3 기울기 소실 때문에 RNN이 앞쪽 단어를 잊는 문제를 줄이려고 나온 것은? (7장) 이름 _____

① CNN ② LSTM ③ TF-IDF

4 TF-IDF 벡터의 코사인 유사도로는 "휘발유"와 "가솔린"이 든 두 문서를 비슷하다고 보기 어렵다. (O / X) (11장, 24-02) 이름 _____

5 처음 보는 단어(OOV) 문제를 푸는 방법은? (15장) 이름 _____

① 불용어 제거 ② 서브워드 토큰화 ③ 원-핫 인코딩

6 seq2seq에서 인코더가 디코더에 넘기는 벡터 하나를 _____ 벡터라고 한다. (16장) 이름 _____

7 트랜스포머에 포지셔널 인코딩이 필요한 이유는? (18장) 이름 _____

① 단어 집합을 줄이려고 ② RNN이 없어 순서 정보가 없어서 ③ 과적합을 막으려고

8 BERT는 다음 단어 맞히기로 사전 훈련한다. (O / X) (19장) 이름 _____

9 개체명 인식처럼 토큰마다 답을 내는 문제에서 BERT의 출력층은 어디에 붙는가? (14장, 19-02) 이름 _____

① [CLS] 벡터 하나 ② 모든 토큰의 벡터 ③ 마지막 토큰의 벡터 하나

10 GPT가 뒤쪽 토큰을 보지 못하게 하는 것은 _____ 셀프 어텐션이다. (18, 21장) 이름 _____

11 RAG는 LLM의 가중치를 내 문서로 다시 학습시키는 방법이다. (O / X) (24장) 이름 _____

12 RAG에서 문서를 작은 조각으로 자르는 일을 _____이라고 한다. (24-03) 이름 _____

13 리랭킹에서 크로스 인코더를 처음부터 모든 문서에 쓰지 않는 이유는? (25-02) 이름 _____

① 정확도가 낮아서 ② 문서마다 모델을 다시 돌려야 해 느려서 ③ 한국어를 못 읽어서

14 "영화 리뷰가 긍정인지 부정인지 맞히기"를 13장의 방법과, 사전 훈련된 BERT를 파인 튜닝하는 방법(19-02)으로 각각 어떻게 푸는지, 차이를 설명해 보세요. 이름 _____

15 "우리 동아리 회칙 PDF를 보고 답하는 챗봇"을 만든다면 어떤 단계가 필요한지, 각 단계에서 쓰는 기술의 장 번호와 함께 말해 보세요. 이름 _____

정답과 해설

종합 문제

1. 손실
2. X — 여럿 중 하나를 고를 때는 소프트맥스입니다.
3. ②
4. O — 글자가 다르면 다른 단어로 셉니다. 임베딩을 쓰는 시맨틱 검색은 찾습니다.
5. ②
6. 컨텍스트 (문맥)
7. ②
8. X — 가린 단어 맞히기(MLM)와 다음 문장 예측(NSP)입니다. 다음 단어 맞히기는 GPT입니다.
9. ②
10. 마스크드 (룩어헤드 마스크를 씌운)
11. X — 가중치는 그대로 두고, 찾은 문서를 프롬프트에 넣습니다.
12. 청킹
13. ②
14. 예시 답: 13장은 단어 집합을 직접 만들고 임베딩 층과 LSTM을 리뷰 데이터만으로 처음부터 학습시킨 뒤 마지막 은닉 상태를 선형층에 넣는다. BERT를 쓰면 이미 언어를 학습한 BERT와 그 토크나이저를 내려받아 [CLS] 벡터에 선형층을 붙이고 작은 학습률로 몇 에포크만 학습시킨다. 뒤쪽은 데이터가 훨씬 적어도 된다.
15. 예시 답: ① PDF 로더로 글을 뽑고 군더더기를 정리한다(24-05). ② 청크로 자른다(24-03). ③ 문장 임베딩 모델로 벡터로 바꿔(19-05) 벡터 데이터베이스에 넣는다(24-06). ④ 질문이 오면 질문을 임베딩해 가까운 청크를 찾고, 필요하면 리랭킹한다(25장). ⑤ 지시, 찾은 청크, 질문을 프롬프트에 넣어 LLM이 답하게 한다(21장, 24-07).

자료 목록

날짜	장	파일 (저장소의 pdf 폴더)
10/6	3장 머신 러닝 입문	03_머신러닝입문.pdf
10/6	4장 로지스틱 회귀	04_로지스틱회귀.pdf
10/6	5장 소프트맥스 회귀	05_소프트맥스회귀.pdf
10/7	6장 인공 신경망	06_인공신경망.pdf
10/7	7장 순환 신경망	07_순환신경망.pdf
10/7	8장 합성곱 신경망	08_합성곱신경망.pdf
10/8	9장 자연어 처리 기초	09_자연어처리기초.pdf
10/8	10장 언어 모델	10_언어모델.pdf
10/12	11장 텍스트 유사도	11_텍스트유사도.pdf
10/12	12장 텍스트 임베딩	12-1_워드임베딩.pdf, 12-2_임베딩활용.pdf
10/13	13장 텍스트 분류	13_텍스트분류.pdf
10/13	14장 시퀀스 레이블링	14_시퀀스레이블링.pdf
10/13	15장 서브워드 토크나이저	15_서브워드토크나이저.pdf
10/14	16장 시퀀스투시퀀스	16_시퀀스투시퀀스.pdf
10/14	17장 어텐션	17_어텐션.pdf
10/15	18장 트랜스포머	18_트랜스포머.pdf
10/16	19장 BERT	19_BERT.pdf
10/16	21장 GPT 학습하기	21_GPT.pdf
10/16	24장 랭체인으로 만드는 RAG	24_RAG.pdf
10/16	25장 고급 RAG 기법	25_고급RAG.pdf